

Politechnika Wrocławska
Sztuczna Inteligencja i Inżynieria Wiedzy

Raport 1

Wyszukiwanie najkrótszych ścieżek i problem komiwojażera w sieci kolejowej

Dawid Pilarski
Numer indeksu: 280468

Wrocław, 2026

1. Wprowadzenie

Tematem tego zadania było zaimplementowanie algorytmów do wyszukiwania tras w sieci kolejowej Kolei Dolnośląskich. Całość opiera się na rzeczywistych danych rozkładowych w formacie GTFS, co sprawia, że problem nie jest czysto teoretyczny, trzeba było zmierzyć się z brudnymi danymi, czasami przekraczającymi północ i skomplikowaną strukturą stacji z wieloma peronami.

Zadanie dzieli się na dwie części. Pierwsza to klasyczne wyszukiwanie najkrótszej ścieżki z przystanku A do B (algorytmy Dijkstry i A*). Druga to wariant problemu komiwojażera, czyli mając listę przystanków do odwiedzenia, szukamy optymalnej kolejności objazdu i powrotu na start (Tabu Search). W obu częściach użytkownik decyduje, czy zależy mu na minimalizacji czasu przejazdu czy liczby przesiadek.

2. Tło teoretyczne

2.1. Problem optymalizacyjny

W skrócie: mamy zbiór możliwych rozwiązań (np. wszystkie możliwe trasy z A do B), z których część spełnia ograniczenia (pociągi faktycznie kursują, zdążymy na przesiadkę) to są rozwiązania dopuszczalne. Każde rozwiązanie oceniamy funkcją celu (czas przejazdu albo liczbą przesiadek) i szukamy tego najlepszego. Najlepsze spośród wszystkich dopuszczalnych to optimum globalne.

2.2. Heurystyka i aproksymacja

Gdy dokładne rozwiązanie jest zbyt kosztowne obliczeniowo, korzystamy z metod przybliżonych. Algorytm aproksymacyjny daje matematyczną gwarancję, że wynik nie odbiega od optimum o więcej niż ϵ . Heurystyka takiej gwarancji nie daje, ale w praktyce zwykle działa dobrze i jest szybsza. W tym zadaniu heurystyki występują w dwóch kontekstach: jako składnik A* (szacowanie odległości do celu) oraz w Tabu Search (cała metoda jest heurystyką).

2.3. Algorytm Dijkstry

Dijkstra znajduje najkrótszą ścieżkę od jednego źródła do wszystkich wierzchołków w grafie z nieujemnymi wagami. Działa jak rozlewająca się woda, zaczyna od startu, rozchodzi się falowo we wszystkich kierunkach, zawsze wybierając wierzchołek z najmniejszą dotychczasową odległością. Gwarantuje znalezienie optimum, ale jest „ślepy” nie wie, w którą stronę leży cel, więc eksploruje niepotrzebne rejony grafu.

W naszym grafie GTFS jest pewien niuans: wagi krawędzi zależą od czasu (ten sam odcinek o 8:00 i o 14:00 może mieć inne pociągi). Dlatego algorytm nie operuje na statycznych wagach, tylko szuka odjazdów późniejszych niż aktualny czas dotarcia.

2.4. Algorytm A*

A* to Dijkstra z „kompasem”. Do kosztu dotarcia $g(n)$ dodaje szacunek odległości do celu $h(n)$, dzięki czemu priorytetyzuje wierzchołki leżące w kierunku celu zamiast eksplorować graf równomiernie.

Warunek: heurystyka musi być dopuszczalna (admissible), czyli nigdy nie przeszacować rzeczywistego kosztu.

Zaimplementowałem dwie heurystyki:

- **Dla kryterium czasu** - odległość geodezyjna (wzór haversine) podzielona przez 160 km/h. To celowo zawyża prędkość, żaden pociąg KD tyle nie rozwija, więc heurystyka zawsze zaniża prawdziwy czas, co gwarantuje dopuszczalność.
- **Dla kryterium przesiadek** - zwraca 0, jeśli z danego przystanku jedzie bezpośredni kurs do celu, albo 1 w przeciwnym razie. Nigdy nie przeszacowuje, bo jeśli mówi „1 przesiadka”, to w rzeczywistości może być 1, 2 czy 5.

2.5. Problem komiwojażera (TSP)

Klasyczny problem optymalizacyjny: mamy zbiór miast (przystanków) i szukamy najkrótszej zamkniętej trasy odwiedzającej każde z nich dokładnie raz. Problem jest NP-trudny dla 10 przystanków to 3,6 miliona permutacji, dla 15 już ponad bilion. Dlatego stosujemy metaheurystyki zamiast przeszukiwania wyczerpującego.

2.6. Tabu Search

Tabu Search to ulepszenie przeszukiwania lokalnego. Zwykle przeszukiwanie lokalne utknie w optimum lokalnym (jest na szczycie pagórka, ale nie widzi wyższej góry za doliną). Tabu Search rozwiązuje to na trzy sposoby:

- **Lista tabu** - zabrania powrotu do niedawno odwiedzonych rozwiązań. Implementowana jako kolejka FIFO o ograniczonym rozmiarze - najstarsze zakazy są usuwane, żeby nie blokować zbyt dużo opcji.
- **Kryterium aspiracji** - wyjątek od reguły. Jeśli ruch jest zakazany, ale prowadzi do najlepszego dotychczas wyniku, to go dopuszczamy. Byłoby głupotą odrzucać świetne rozwiązanie tylko dlatego, że jest na liście tabu.
- **Próbkowanie sąsiedztwa** - zamiast sprawdzać każdego sąsiada (co przy dużych instancjach jest wolne), losowo wybieramy podzbiór do oceny.

3. Dane GTFS i konstrukcja grafu

3.1. Źródło danych

Dane pochodzą od Kolei Dolnośląskich i obejmują okres 3.03.2026-12.12.2026. Format GTFS to zestaw plików CSV opisujących sieć transportową: agency.txt (przewoźnik), routes.txt (57 tras, np. D1, D6, D66), stops.txt (1108 przystanków i peronów), trips.txt (3260 kursów), stop_times.txt (48727 czasów na przystankach), calendar.txt i calendar_dates.txt (kalendarze kursowania i wyjątki).

3.2. Budowa grafu

Graf buduję dynamicznie dla wybranej daty. Proces wygląda tak:

1. Każdy peron (stop z location_type=0) staje się wierzchołkiem grafu ze współrzędnymi GPS, które później wykorzystuje heurystyka A*.
2. Filtruję kursy względem dnia tygodnia (calendar.txt) z uwzględnieniem wyjątków z calendar_dates.txt - np. w święto kurs może być dodany lub odwołany.
3. Dla każdego aktywnego kursu tworzę krawędzie między kolejnymi przystankami (wg stop_sequence) z czasem odjazdu i przyjazdu.
4. Perony tej samej stacji (łączę je przez parent_station) są połączone krawędziami o zerowym koszcie - modeluje to fizyczne przejście po stacji przy przesiadce.

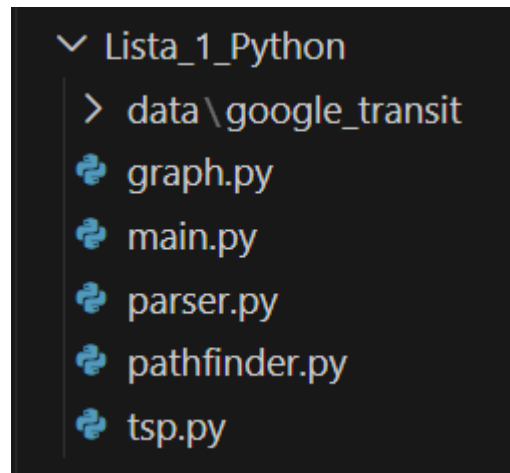
Na końcu odjazdu z każdego przystanku są sortowane po czasie, co pozwala na szybkie wyszukiwanie binarne (bisect) najbliższego odjazdu.

4. Implementacja

4.1. Struktura projektu

Całość jest podzielona na pięć plików, każdy z jasną odpowiedzialnością:

- `parser.py` - wczytuje pliki GTFS do pamięci. Klasa `GTFSData` trzyma słowniki przystanków, tras, kursów, czasów i kalendarzy. Obsługuje też normalizację nazw (polskie znaki, wielkość liter).
- `graph.py` - buduje graf zależny od czasu. Funkcja `build_graph` zwraca słownik: `stop_id` → posortowana lista odjazdów.
- `pathfinder.py` - implementacja A*/Dijkstry. Dijkstra to A* z heurystyką $h=0$ oba algorytmy są jednym kodem różniącym się tylko argumentem.
- `tsp.py` - Tabu Search w czterech wariantach (basic, variable, aspiration, sampling).
- `main.py` - punkt wejścia, parsowanie argumentów z linii poleceń, uruchamianie odpowiednich funkcji.



4.2. Zadanie 1 - Wyszukiwanie ścieżki $A \rightarrow B$

a) Algorytm Dijkstry - kryterium czasu

Dijkstra jest zaimplementowany jako szczególny przypadek A* z heurystyką zawsze zwracającą 0. W kodzie wygląda to tak, że funkcja `find_path` przyjmuje heurystykę jako argument, podanie `zero_heuristic` zamienia A* w czystego Dijkstrę. Dzięki temu uniknąłem duplikacji kodu.

Kolejka priorytetowa (`heapq`) w każdej iteracji wyciąga stan o najniższym koszcie. Dla kryterium czasu kosztem jest czas od momentu startu do bieżącego wierzchołka.

```
PS C:\Users\Dell\Desktop\AI\Lista_1_Python\Lista_1_Python> python main
.py "Wałbrzych Główny" "Lubin" t 08:00:00 2026-03-10 dijkstra
Cel: Lubin (4 peronow)
Kryterium: czas
Algorytm: dijkstra
Wałbrzych Główny | Wrocław Główny | D60 | 08:18:00 | 09:39:00
  --- Przesiadka (6min oczekiwania) ---
Wrocław Główny | Wrocław Muchobór | D1 | 09:45:00 | 09:50:00
  --- Przesiadka (11min oczekiwania) ---
Wrocław Muchobór | Rudna Miasto | D2 | 10:01:00 | 11:16:00
  --- Przesiadka (3min oczekiwania) ---
Rudna Miasto | Lubin | D12 | 11:19:00 | 11:32:00
--- Wynik ---
Rudna Miasto | Lubin | D12 | 11:19:00 | 11:32:00
--- Wynik ---
Kryterium: czas przejazdu
Całkowity czas: 3h 32min (12720 sek.)
Przesiadki: 3
Czas obliczeń: 9 ms
Węzły rozwinięte: 718
PS C:\Users\Dell\Desktop\AI\Lista_1_Python\Lista_1_Python>
```

Dijkstra znalazł trasę Wałbrzych Główny do Lubin w 3h 32min z 3 przesiadkami (przez Wrocław, Muchobór i Rudną Miasto). Rozwinął 718 węzłów w 9 ms.

b) Algorytm A* - kryterium czasu

Heurystyka haversine oblicza odległość geodezyjną między bieżącym przystankiem a celem i dzieli ją przez 160 km/h, dając dolne oszacowanie czasu w sekundach. Dzięki temu algorytm priorytetyzuje przystanki leżące bliżej celu i nie traci czasu na eksplorowanie odległych odgałęzień grafu.

Pruning (odcinanie gałęzi) działa tak: jeśli dotarliśmy do przystanku X z kosztem 5000, a teraz docieramy z kosztem 6000, to ta gorsza ścieżka jest pomijana.

```
PS C:\Users\Dell\Desktop\AI\Lista_1_Python\Lista_1_Python> python main.py "Wałbrzych Główny" "Lub
in" t 08:00:00 2026-03-10 astar
Ladowanie GTFS z: data/google_transit
Zaladowano: 1108 przystankow, 57 tras, 3263 kursow, 48748 stop_times
Graf: 487 przystankow z odjazdami, 9191 polaczen
Start: Wałbrzych Główny (4 peronow)
Cel: Lubin (4 peronow)
Kryterium: czas
Algorytm: astar
Wałbrzych Główny | Wrocław Główny | D60 | 08:18:00 | 09:39:00
--- Przesiadka (6min oczekiwania) ---
Wrocław Główny | Wrocław Muchobór | D1 | 09:45:00 | 09:50:00
--- Przesiadka (11min oczekiwania) ---
Wrocław Muchobór | Rudna Miasto | D2 | 10:01:00 | 11:16:00
--- Przesiadka (3min oczekiwania) ---
Rudna Miasto | Lubin | D12 | 11:19:00 | 11:32:00
--- Wynik ---
Kryterium: czas przejazdu
Calkowity czas: 3h 32min (12720 sek.)
Przesiadki: 3
Czas obliczen: 12 ms
Wzly rozwiniete: 659
```

A* znalazł identyczną trasę (3h 32min, 3 przesiadki), ale rozwinął 659 węzłów zamiast 718, czyli o 8% mniej niż Dijkstra. Heurystyka haversine pozwoliła pominąć przystanki leżące w przeciwnym kierunku od celu.

c) Algorytm A*- kryterium przesiadek

Przy minimalizacji przesiadek kosztem g jest liczba dotychczasowych przesiadek. Heurystyka sprawdza, czy z danego przystanku jedzie bezpośredni kurs do celu jeśli tak, zwraca 0, w przeciwnym razie 1.

Kluczowa różnica względem kryterium czasu: klucz pruningu to $(stop_id, last_trip_id)$ zamiast samego $stop_id$. Dlaczego? Bo ten sam przystanek osiągnięty innym pociągami to faktycznie inny stan - może prowadzić do mniejszej liczby przesiadek dalej. Początkowo tego nie miałem i algorytm znajdował nieoptymalne rozwiązania.

```
PS C:\Users\Dell\Desktop\AI\Lista_1_Python\Lista_1_Python> python main.py "Wałbrzych Główny" "Lubin" p 08:00:00 2026-03-10 astar
Ładowanie GTFS z: data/google_transit
Zaladowano: 1108 przystankow, 57 tras, 3263 kursow, 48748 stop_times
Graf: 487 przystankow z odjazdami, 9191 polaczen
Start: Wałbrzych Główny (4 peronow)
Cel: Lubin (4 peronow)
Kryterium: przesiadki
Algorytm: astar
Wałbrzych Główny | Wrocław Główny | D6 | 09:31:00 | 10:44:00
--- Przesiadka (1h 32min oczekiwania) ---
Wrocław Główny | Lubin | D11 | 12:16:00 | 13:10:00
--- Wynik ---
Kryterium: liczba przesiadek
Całkowity czas: 5h 10min
Przesiadki: 1
Czas obliczeń: 4 ms
Węzły rozwinięte: 383
PS C:\Users\Dell\Desktop\AI\Lista_1_Python\Lista_1_Python>
```

Przy kryterium przesiadek algorytm znalazł zupełnie inną trasę - tylko 1 przesiadka (D6 do Wrocławia, potem D11 do Lubina), ale kosztem dłuższego czasu: 5h 10min zamiast 3h 32min. To dobrze pokazuje, że algorytm faktycznie optymalizuje wybrane kryterium.

d) Modyfikacje A*

Oprócz samej heurystyki, zastosowałem kilka optymalizacji, które znacząco wpłynęły na wydajność:

- **Wyszukiwanie binarne odjazdów** - odjazdy z każdego przystanku są posortowane, więc szukanie najbliższego odjazdu po danej godzinie to $O(\log n)$ dzięki `bisect_left` z modułu `bisect`. Bez tego byłoby liniowe $O(n)$.
- **Limity eksploracji** - maksymalnie 500 połączeń na przystanek i 4h oczekiwania. Bez tego duże stacje (Wrocław Główny ma mnóstwo odjazdów) generowały eksplozję stanów.
- **Deduplikacja kursów** - z danego przystanku bierzemy tylko pierwszy odjazd każdego tripu, pomijając kolejne wpisy tego samego kursu.

4.3. Zadanie 2 - Problem komiwojażera

Każdy wariant Tabu Search korzysta z klasy TSPSolver, która jako „czarną skrzynkę” używa A* do obliczania kosztu przejazdu między parami przystanków. Wyniki są cache’owane - raz obliczony koszt Wrocław do Legnica o 10:00 nie wymaga ponownego A*. Sąsiedztwo definiuje operacja 2-opt (odwrócenie fragmentu permutacji).

a) Wariant podstawowy - tabu bez limitu

Najprostsza wersja. Startujemy od losowej permutacji, w każdej iteracji sprawdzamy całe sąsiedztwo 2-opt i przechodzimy do najlepszego sąsiada, który nie jest na liście tabu. Lista tabu rośnie bez ograniczeń - każda odwiedzona permutacja jest zakazana na zawsze. Zaleta: nigdy nie wrócimy do tego samego rozwiązania. Wada: możemy się zablokować.

b) Zmienny rozmiar T

Różnica: lista tabu ma ograniczony rozmiar $t = \max(5, n^2/2)$, gdzie n to liczba przystanków. Implementacja przez deque (kolejka FIFO) + set (szybkie sprawdzanie). Gdy tabu się przepełni, najstarszy wpis jest usuwany. Im więcej przystanków, tym większa tabu - bo większe sąsiedztwo wymaga dłuższej pamięci, żeby uniknąć cykli.

c) Kryterium aspiracji

Dochodzi jeden warunek: rozwiązanie z listy tabu jest dopuszczalne, jeśli jego koszt jest lepszy od globalnego optimum. W kodzie to jedna linijka: jeśli is_tabu AND $koszt \geq best_cost \rightarrow$ pomiń. W przeciwnym razie ($koszt < best_cost$) $\rightarrow$ dopuść mimo zakazu.

d) Próbkowanie sąsiedztwa

Dwie zmiany. Po pierwsze, zamiast przeglądać całe sąsiedztwo 2-opt, losowo próbkujemy $\max(10, 2n)$ par. Po drugie, dodajemy operacje insert (wyjęcie elementu z jednej pozycji i wstawienie w inną). Dwa typy ruchów pozwalają na bardziej zróżnicowaną eksplorację. Deterministyczny seed Random(42) zapewnia powtarzalność.

```
PS C:\Users\Dell\Desktop\AI\Lista_1_Python\Lista_1_Python> python main.py --tsp "Wrocław Główny"
"Legnica;Jelenia Góra;Wałbrzych Główny" t 06:00:00 2026-03-10 aspiration
● Ładowanie GTFS z: data/google_transit
Zaladowano: 1108 przystankow, 57 tras, 3263 kursow, 48748 stop_times
Graf: 487 przystankow z odjazdami, 9191 polaczen
TSP: start=Wrocław Główny, przystanki=['Legnica', 'Jelenia Góra', 'Wałbrzych Główny']
Kryterium: czas
Wariant: aspiration
--- Wynik TSP ---
Kolejnosc: Wrocław Główny -> Jelenia Góra -> Wałbrzych Główny -> Legnica -> Wrocław Główny
Całkowity czas: 5h 46min (20760 sek.)
Czas obliczeń: 146 ms
Wrocław Główny | Jelenia Góra | D6 | 06:10:00 | 08:27:00
--- Przesiadka (22min oczekiwania) ---
Jelenia Góra | Wałbrzych Główny | D6 | 08:49:00 | 09:30:00
--- Przesiadka (1min oczekiwania) ---
Wałbrzych Główny | Jaworzyna Śląska | D6 | 09:31:00 | 10:07:00
--- Przesiadka (5min oczekiwania) ---
Jaworzyna Śląska | Legnica | D64/D91 | 10:12:00 | 10:58:00
--- Przesiadka (9min oczekiwania) ---
Legnica | Wrocław Główny | D10 | 11:07:00 | 11:46:00
```

Przy kryterium przesiadek algorytm znalazł zupełnie inną trasę - tylko 1 przesiadka (D6 do Wrocławia, potem D11 do Lubina), ale kosztem dłuższego czasu: 5h 10min zamiast 3h 32min. To dobrze pokazuje, że algorytm faktycznie optymalizuje wybrane kryterium.

```

Wariant: basic
Kolejnosc: Wroclaw Główny -> Jelenia Góra -> Walbrzych Główny -> Legnica -> Wroclaw Główny
Całkowity czas: 5h 46min (20760 sek.)
Czas obliczeń: 138 ms

=== WARIANT: variable ===
Wariant: variable
Kolejnosc: Wroclaw Główny -> Jelenia Góra -> Walbrzych Główny -> Legnica -> Wroclaw Główny
Całkowity czas: 5h 46min (20760 sek.)
Czas obliczeń: 150 ms

=== WARIANT: aspiration ===
Wariant: aspiration
Kolejnosc: Wroclaw Główny -> Walbrzych Główny -> Jelenia Góra -> Legnica -> Wroclaw Główny
Całkowity czas: 5h 46min (20760 sek.)
Czas obliczeń: 123 ms

=== WARIANT: sampling ===
Wariant: sampling
Kolejnosc: Wroclaw Główny -> Jelenia Góra -> Walbrzych Główny -> Legnica -> Wroclaw Główny
Całkowity czas: 5h 46min (20760 sek.)
Czas obliczeń: 134 ms

```

Wszystkie cztery warianty znalazły ten sam koszt (5h 46min), co przy 3 przystankach jest oczekiwane - przestrzeń rozwiązań jest na tyle mała, że każdy wariant szybko zbiega do optimum. Różnice w wariantach byłyby widoczne przy większej liczbie przystanków. Najszybszy był wariant aspiration (123 ms), najwolniejszy variable (150 ms).

5. Użyte biblioteki

Projekt korzysta wyłącznie z biblioteki standardowej Pythona, bez żadnych zależności zewnętrznych:

- csv - wczytywanie plików GTFS (format CSV)
- heapq - kolejka priorytetowa (min-heap) dla algorytmów Dijkstry i A*
- math - obliczenia trygonometryczne we wzorze haversine
- bisect - wyszukiwanie binarne w posortowanych listach odjazdów
- random - losowe permutacje i próbkowanie w Tabu Search
- collections.deque - kolejka FIFO dla listy tabu z ograniczonym rozmiarem
- datetime - obsługa dat i weryfikacja kalendarzy kursowania
- time - pomiar czasu obliczeń
- sys, os - obsługa argumentów wiersza poleceń i ścieżek plików

6. Napotkane problemy

- **Czasy po północy** - GTFS dopuszcza wartości $> 23:59:59$ (np. $25:10:00 = 01:10$ następnego dnia). Na początku powodowało to błędy w obliczeniach. Rozwiązanie: operuję na sekundach od północy bez normalizacji do 0-86400.
- **Przesiadki między peronami** - rozróżnienie między stacją (`location_type=1`) a peronem (`location_type=0`) oraz poprawne użycie `parent_station` było kluczowe. Bez tego program nie łączył peronów i przesiadki nie działały.
- **Wydajność TSP** - każda ewaluacja permutacji odpala wielokrotnie A^* , co jest kosztowne. Cache kosztów i ścieżek rozwiązał problem częściowo, ale dla długich list czas obliczeń jest wciąż znaczący.

7. Wykorzystanie narzędzi AI

Podczas pracy nad zadaniem korzystałem z narzędzia AI Gemini. Szacuję udział kodu wygenerowanego przez AI na ok. 70%. AI pomogło mi w:

- generowaniu kodu parsującego pliki GTFS i budującego struktury danych,
- implementacji szkieletów algorytmów A^* /Dijkstra i Tabu Search,
- optymalizacjach takich jak binary search, cache kosztów i pruning.

Samodzielnie zaprojektowałem podział na moduły i ogólną architekturę rozwiązania, dobrałem heurystyki (haversine z prędkością 160 km/h, heurystyka przesiadkowa), naprawiłem błędy w wygenerowanym kodzie (m.in. niepoprawny klucz pruningu przy kryterium przesiadek), przetestowałem rozwiązanie na rzeczywistych danych KD i dostosowałem parametry algorytmów. Analiza treści zadania, zrozumienie algorytmów na podstawie wykładów i materiałów oraz interpretacja wyników to moja praca.

8. Wnioski

Oba algorytmy (Dijkstra i A^*) znajdują identyczne rozwiązania, co potwierdza poprawność implementacji. A^* rozwija mniej węzłów dzięki heurystyce (659 vs 718 dla trasy Wałbrzych Główny - Lubin), choć przy małej sieci KD różnica w czasie obliczeń jest niewielka rzędu kilku milisekund. Przy większych sieciach (np. dane GTFS z kilku przewoźników) przewaga A^* byłaby bardziej widoczna.

Trasa Wałbrzych Główny - Lubin dobrze pokazuje różnicę między kryteriami: optymalizacja czasu daje 3h 32min z 3 przesiadkami, a optymalizacja przesiadek 5h 10min z jedną. Algorytm poprawnie realizuje oba cele.

W przypadku Tabu Search, wszystkie cztery warianty zbiegły do tego samego kosztu (5h 46min) dla 3 przystanków. Przy tak małej instancji przestrzeń rozwiązań jest na tyle ograniczona, że różnice między wariantami nie są widoczne. Modyfikacje (zmienny rozmiar T, aspiracja, próbkowanie) pełnią rolę zabezpieczeń, których przewaga ujawnia się dopiero przy większej liczbie przystanków.